

Production Order List Query — Function Design (機能設計)

002_DD-FN — used by 002_DD and 002_DD-API, requirements REQ-020–REQ-027.

Document control (改版履歴)

Field	Value
Document ID	002_DD-FN
System name	ProductionManagementAI
Subsystem name	Production orders
Work item	WI-003
Created by	Claude (for ThanhTN)
Created date	2026-09-22
Last updated by	Claude (for ThanhTN)
Last updated date	2026-09-22

Version	Date	Author	Revision content
1	2026-09-22	Claude (for ThanhTN)	Initial creation
2	2026-09-23	Claude (for ThanhTN)	WI-005: the UI is Japanese. The one quoted UI text gives the Japanese paging label with an English gloss (WI-005 DEC-003). No field, rule, API or processing changes

Overview and method index

Field	Value
Module name	<code>ProductionOrderService.ListAsync</code> and the repository methods behind it (<code>ProductionManagementAI.Application.ProductionOrders</code> , implemented in <code>ProductionManagementAI.Infrastructure.ProductionOrders</code>)

Field	Value
Overview	The read side of production orders: turn a validated list query into a counted page of rows. Extends the existing <code>ProductionOrderService</code> and <code>IProductionOrderRepository</code> from 001_DD-FN rather than introducing a second service, so the plant clock, telemetry and result conventions stay shared

No	Method name	Overview	Notes
1	<code>ProductionOrderService.ListAsync</code>	Validate the product filter, count matches, fetch the page, mark overdue rows	API-PO-04
2	<code>IProductionOrderRepository.CountOrdersAsync</code>	<code>count(*)</code> over the filtered set	New
3	<code>IProductionOrderRepository.ListOrdersAsync</code>	One ordered, projected page of rows	New
4	<code>ProductionOrderQueryExtensions.ApplyFilters</code>	Compose only the filters that are set onto an <code>IQueryable</code>	New, internal
5	<code>ProductionOrderQueryExtensions.ApplySort</code>	Map the allow-listed sort key to an ordering, with the order-number tie-breaker	New, internal
6	<code>ProductionOrderListMapper.ToListItem</code>	Row → <code>ProductionOrderListItem</code> , computing <code>isOverdue</code>	New, internal
7	<code>IPlantClock.Today</code>	Plant-local date (<code>Asia/Tokyo</code>) used for <code>isOverdue</code>	Existing (001_DD-FN §6), reused unchanged
8	<code>IProductionOrderRepository.ProductExistsAsync</code>	Product-filter existence check	Existing (001_DD-FN), reused unchanged

`ListAsync` returns the same `Result<T>` type as 001_DD-FN's methods, using only the `Ok` and `Invalid` variants — a list query cannot be `NotFound`, `Conflict` or a `RuleViolation`. Unexpected exceptions propagate to the global Problem Details handler (500, MSG-E013) rather than becoming a `Result`.

Shared utility references

No	Name	Overview	Notes
1	<code>IProductionOrderRepository</code>	EF Core data access; gains the two read methods above	Existing interface (001_DD-FN)
2	<code>IPlantClock</code>	Plant-local date and year	Existing; <code>FakeTimeProvider</code> in tests makes <code>isOverdue</code> deterministic
3	<code>ProductionOrderTelemetry</code>	<code>ActivitySource</code> + <code>Meter</code> named <code>ProductionManagementAI.ProductionOrders</code>	Existing; gains one counter and one histogram (see Observability)
4	<code>ILogger<ProductionOrderService></code>	Structured logging, <code>LoggerMessage</code> source-generated	Never logs the order-number fragment as free text — it is a user-supplied value and goes in as a structured field only
5	<code>ProductionOrderListQuery</code>	The validated query object the API layer produces	Defined in 002_DD module 5; this module never sees raw query strings

Request data

Parameters this module sends to the database. All are bound parameters; nothing is concatenated into SQL (`ai/rules/database.md`).

No	Name	Variable name	Type	Length	Used by (which method(s))	Notes
1	Status set	<code>statuses</code>	<code>ProductionOrderStatus[]</code>	≤ 4	ListOrdersAsync, CountOrdersAsync	<code>status = ANY(@statuses)</code> ; omitted entirely when empty
2	Product ID	<code>productId</code>	<code>Guid?</code>	—	ListOrdersAsync, CountOrdersAsync, ProductExistsAsync	Omitted when null
3	Due from	<code>dueFrom</code>	<code>DateOnly?</code>	—	ListOrdersAsync, CountOrdersAsync	<code>due_date >= @dueFrom</code>
4	Due to	<code>dueTo</code>	<code>DateOnly?</code>	—	ListOrdersAsync, CountOrdersAsync	<code>due_date <= @dueTo</code>
5	Order-number pattern	<code>pattern</code>	<code>string?</code>	≤ 24	ListOrdersAsync, CountOrdersAsync	Already trimmed, upper-cased, escaped and wrapped as <code>%...%</code> by the API layer (002_DD-API); used with <code>EF.Functions.Like(o.OrderNumber, pattern, "\\")</code>
6	Skip / take	<code>skip</code> , <code>take</code>	<code>int</code>	—	ListOrdersAsync	<code>(page - 1) × pageSize</code> , <code>pageSize</code>

Response / value mapping

`ProductionOrderListItem` (field catalog in 002_DD-API) is built by `ProductionOrderListMapper.ToListItem` from the projected row:

No	Name	Variable name	Type	Length	Required	Value mapping	Notes
1	ID	<code>Id</code>	<code>Guid</code>	—	yes	as stored	
2	Order number	<code>OrderNumber</code>	<code>string</code>	13	yes	as stored (generated column)	
3	Product	<code>Product</code>	<code>ProductSummary</code>	—	yes	<code>{ p.Id, p.Sku, p.Name }</code> from the join	Narrower than <code>ProductResponse</code> , but the same three fields

No	Name	Variable name	Type	Length	Required	Value mapping	Notes
4	Quantity	Quantity	int	—	yes	as stored	
5	Due date	DueDate	DateOnly	—	yes	as stored	Serialized YYYY-MM-DD , no timezone conversion
6	Status	Status	string	—	yes	row.Status.ToString()	
7	Overdue	IsOverdue	bool	—	yes	row.DueDate < plantToday && (row.Status == Draft row.Status == InProgress)	Computed in memory over the page's rows, after the query — never stored, never a SQL predicate (002_DB)
8	Updated at	UpdatedAt	DateTimeOffset	—	yes	updated_at_utc	UTC

`PagedResult<T>` wraps the items with `Total` , `Page` , `PageSize` , `Sort` and `Dir` echoed from the validated query.

Method design

1. `ProductionOrderService.ListAsync`

Field	Value
Description	Return one counted page of production orders for a validated list query
Return type	<code>Task<Result<PagedResult<ProductionOrderListItem>>></code>
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments

No	Type	Name	Description
1	<code>ProductionOrderListQuery</code>	<code>query</code>	Already validated and normalized by the API layer (002_DD module 5)
2	<code>CancellationToken</code>	<code>cancellationToken</code>	

Return value

Type	Name	Description
<code>Result<PagedResult<ProductionOrderListItem>></code>	result	<code>Ok</code> with the page, or <code>Invalid</code> with <code>productId</code> → MSG-E002

Processing overview: check the product filter exists (the one validation that needs the database), count the matches, fetch the page only when the count is non-zero, then map the rows and mark the overdue ones against the plant clock. The count runs first so a page past the last one costs one cheap query instead of two, and so `total` is always present for the summary.

Processing flow

Step	Description	Calls
1	Start the span <code>production_orders.list</code> with the filter shape as attributes (which filters are set, sort key, direction, page, page size — never the fragment text)	<code>ProductionOrderTelemetry.Source</code>
2	If <code>query.ProductId</code> is set and <code>ProductExistsAsync</code> is false → return <code>Invalid(productId: MSG-E002)</code> ; record outcome <code>validation_failed</code>	<code>IProductionOrderRepository.ProductExistsAsync</code>
3	<code>total = await CountOrdersAsync(query)</code>	\$2
4	If <code>total == 0</code> → skip the page query and use an empty item list	—
5	Else <code>rows = await ListOrdersAsync(query)</code>	\$3
6	<code>today = plantClock.Today</code> ; map each row with <code>ToListItem(row, today)</code>	\$6, <code>IPlantClock</code>
7	Record the result-count histogram and outcome <code>success</code> ; return <code>Ok(new PagedResult(...))</code> with the echoed controls	<code>ProductionOrderTelemetry</code>

A page beyond the last one is not an error (REQ-024): step 5 simply returns no rows, and `total` still reports the real number of matches, so the client can show 「9 / 5 ページ」 (Page 9 of 5) honestly and page back.

2. `IProductionOrderRepository.CountOrdersAsync`

Field	Value
Description	Number of production orders matching the query's filters
Return type	<code>Task<int></code>
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments

No	Type	Name	Description
1	ProductionOrderListQuery	query	Only its filter fields are used; sort and paging are ignored
2	CancellationToken	cancellationToken	

Return value

Type	Name	Description
int	total	Exact count, not an estimate (REQ-024)

Processing overview: `db.ProductionOrders.AsNoTracking()` , filters applied by \$4, then `CountAsync` . No join to `products` : every filter predicate is on `production_orders` , so the count query touches one table (002_DB).

Processing flow

Step	Description	Calls
1	Start from <code>db.ProductionOrders.AsNoTracking()</code>	—
2	Apply the filters that are set	\$4
3	<code>CountAsync</code>	EF Core

3. `IProductionOrderRepository.ListOrdersAsync`

Field	Value
Description	One page of production orders, ordered and projected
Return type	<code>Task<IReadOnlyList<ProductionOrderListRow>></code>
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments

No	Type	Name	Description
1	ProductionOrderListQuery	query	Filters, sort, direction, page, page size
2	CancellationToken	cancellationToken	

Return value

Type	Name	Description
IReadOnlyList<ProductionOrderListRow>	rows	Projected rows in the requested order; at most pageSize

Processing overview: the same filtered IQueryable , joined to products for the displayed product and the product sort key, ordered by \$5, then Skip / Take and an explicit projection into ProductionOrderListRow — notes , xmin , order_year , order_seq and created_at_utc are never selected (002_DB read projection).

Processing flow

Step	Description	Calls
1	Start from db.ProductionOrders.AsNoTracking()	—
2	Apply the filters that are set	\$4
3	Join db.Products on product_id into ProductionOrderJoin , a named type with settable members (no navigation property exists, and EF sees through member-init projections)	—
4	Apply the ordering and the order_number tie-breaker on the joined entities	\$5
5	Skip((page - 1) × pageSize).Take(pageSize)	—
6	Project into ProductionOrderListRow and ToListAsync	—

The order of steps 4 and 6 matters: EF Core cannot translate an ORDER BY that reads a member back out of a constructor projection, so the sort is applied before the final projection, not after it. Proven by the integration tests, which fail with a translation error if the two are swapped.

4. ProductionOrderQueryExtensions.ApplyFilters

Field	Value
Description	Compose onto an IQueryable<ProductionOrder> only the filters the query actually carries
Return type	IQueryable<ProductionOrder>
Created by / date	Claude / 2026-09-22

Field	Value
Last modified by / date	—

Arguments

No	Type	Name	Description
1	IQueryable<ProductionOrder>	source	
2	ProductionOrderListQuery	query	

Return value

Type	Name	Description
IQueryable<ProductionOrder>	filtered	The same source with zero to five predicates composed onto it

Processing overview: each filter is added inside an `if`, so an unset filter contributes no SQL at all. This matters for the query plan — a `(@p IS NULL OR col = @p)` predicate would defeat the indexes 002_DB relies on — and it is the reason this is one shared method used by both the count and the page query, so the two can never drift apart.

Processing flow

Step	Description	Calls
1	<code>if (query.Statuses.Count > 0) → Where(o => query.Statuses.Contains(o.Status))</code> (translated to <code>status = ANY(@statuses)</code>)	—
2	<code>if (query.ProductId is Guid id) → Where(o => o.ProductId == id)</code>	—
3	<code>if (query.DueFrom is DateOnly from) → Where(o => o.DueDate >= from)</code>	—
4	<code>if (query.DueTo is DateOnly to) → Where(o => o.DueDate <= to)</code>	—
5	<code>if (query.OrderNumberPattern is string pattern) → Where(o => EF.Functions.Like(o.OrderNumber, pattern, "\\\"))</code>	<code>pg_trgm</code> index (DEC-010)

5. ProductionOrderQueryExtensions.ApplySort

Field	Value
Description	Translate the allow-listed sort key and direction into an ordering, always ending with the order-number tie-breaker
Return type	IOrderedQueryable<...>

Field	Value
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments

No	Type	Name	Description
1	IQueryable<ProductionOrderJoin>	source	The filtered query joined to products , before the list projection
2	ProductionOrderSort	sort	Enum, not a string — the API layer already rejected anything outside the allow-list
3	SortDirection	dir	Asc or Desc

Return value

Type	Name	Description
IOrderedQueryable<...>	ordered	Deterministic total order

Processing overview: a switch over the enum picks the key expression; the direction picks `OrderBy / OrderByDescending ; ThenBy(o => o.OrderNumber)` is always appended unless the key already is the order number. Because the enum is the only input, no client string ever reaches the ordering — the allow-list is enforced by the type system, not by string comparison at this layer.

Processing flow

Step	Description	Calls
1	DueDate → j.Order.DueDate (the default)	—
2	OrderNumber → j.Order.OrderNumber ; no tie-breaker needed (unique)	—
3	Product → j.Product.Sku , then j.Product.Name	—
4	Quantity → j.Order.Quantity	—
5	Status → the workflow rank expression Draft=1, InProgress=2, Completed=3, Cancelled=4 , not the stored string (002_DB sort-key mapping)	—
6	UpdatedAt → j.Order.UpdatedAtUtc	—
7	Append ThenBy(j => j.Order.OrderNumber) ascending (steps 1, 3–6)	—

The direction applies to the chosen key only; the tie-breaker stays ascending, so a descending page is still a stable total order.

6. ProductionOrderListItemMapper.ToListItem

Field	Value
Description	Turn a projected row into the API list item, deciding the overdue flag
Return type	ProductionOrderListItem
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments

No	Type	Name	Description
1	ProductionOrderListRow	row	
2	DateOnly	plantToday	Read once per request, not per row, so every row on a page judges "today" identically

Return value

Type	Name	Description
ProductionOrderListItem	item	See Response / value mapping above

Processing overview: a pure function — same row plus same date always gives the same item, which is what makes the overdue rule unit-testable without a database (002_BD M-08, FN-013).

Processing flow

Step	Description	Calls
1	Copy Id , OrderNumber , Quantity , DueDate , Status , UpdatedAt	—
2	Build Product from the row's three product columns	—
3	IsOverdue = row.DueDate < plantToday && row.Status is Draft or InProgress	—

Observability

Extends the existing `ProductionManagementAI.ProductionOrders` `ActivitySource` / `Meter` (001_DD-FN) rather than adding a second one, per `ai/rules/backend.md`.

Signal	Name	Type	Attributes / buckets
Span	<code>production_orders.list</code>	Activity	<code>filter.status_count</code> , <code>filter.has_product</code> , <code>filter.has_due_range</code> , <code>filter.has_order_number</code> (booleans/counts only — never the fragment text), <code>sort</code> , <code>dir</code> , <code>page</code> , <code>page_size</code> , <code>result.total</code>
Metric	<code>pmai.production_orders.listed</code>	Counter	<code>outcome = success validation_failed error</code> , reusing 001_DD-FN's outcome names
Metric	<code>pmai.production_orders.list_result_size</code>	Histogram	Number of rows returned on the page — shows whether users actually page, and whether page sizes above 20 are used

Logging: one `Information` record per rejected query with the offending parameter names and their message IDs (structured fields, never an interpolated string), and `Error` with the exception for an unexpected failure. The order-number fragment is a user-supplied value and is logged as a structured field only, never concatenated into a message.

Unresolved decisions

None. The behavior here follows DEC-002 (paging and sort), DEC-005 (multi-select status), DEC-010 (fragment matching) and 002_DB's query design; the plant clock and the `Result<T>` /telemetry conventions are inherited unchanged from 001_DD-FN.